

프로젝트 2. 자유 주제 자동화 설계 및 구현 보고서

스마트스토어 운영자를 위한 일일 업무 알림 자동화 — Google Calendar → Gmail (Make.com)

작성일: 2026. 07. 16.

1. 개요

본 프로젝트는 스마트스토어를 운영하며 매일 반복되는 '상품 소싱 - 상세페이지 작성 - 주문 처리' 등의 업무 일정을 사람이 직접 캘린더를 열어 확인하지 않아도, 그날의 할 일 목록이 자동으로 이메일로 정리되어 도착하도록 만드는 자동화이다. 업무용으로 사용 중인 Google 캘린더에 미리 입력해 둔 일정을 자동화 도구가 매일 아침 조회하여, 시간순으로 정리한 뒤 담당자의 Gmail 로 발송한다. 일정이 없는 날에는 메일 자체가 발송되지 않도록 조건 분기도 함께 구현하였다.

자동화 도구는 Make.com(구 Integromat)을 선정하였으며, Google Calendar - Text Aggregator - Gmail 로 이어지는 4 단계 워크플로우를 구성하였다. 아래에서 과제 목표, 기능 요구사항, 실제 수행 과정과 시행착오, 그리고 최종 결과물을 순서대로 정리한다.

2. 과제 목표

이번 과제를 마친 뒤 아래 내용을 스스로 설명할 수 있는 것을 목표로 하였다.

- 자동화 워크플로우를 구성하는 Trigger(조회 시작점)와 Action(후속 처리 동작)의 개념과 역할
- 여러 데이터를 하나로 합치는 Text Aggregator, 값을 저장해두는 Set variable 등 보조 도구(Tools)의 쓰임
- Make.com 이라는 자동화 도구를 실제로 다뤄보며 그 특징(모듈 연결 방식, 데이터 매핑, 오류 메시지 확인 등)을 이해
- 업무 상황(캘린더 일정 → 이메일 알림)에 적합한 자동화 흐름을 스스로 설계하고, 왜 이 도구와 구조를 선택했는지 설명
- 설계한 자동화 흐름을 단계별로(트리거 → 가공 → 발송) 나누어 설명

3. 기능 요구사항

3.1 공통 요구사항 대비 구현 현황

요구 항목	구현 내용
Trigger 1 개 이상	Google Calendar 'Search Events' 모듈이 매 실행 시 당일 일정을 조회하는 시작점 역할 수행
Action 2 개 이상	① Set variable(시간 포맷 변환) ② Text Aggregator(일정 목록 병합) ③ Gmail(이메일 발송) — 총 3 개 Action 구현
조건 분기 (Filter/Router)	별도의 Filter/Router 모듈을 추가하는 대신, Text Aggregator(Tools)의 'Stop processing after an empty aggregation' 옵션을 'Yes'로 설정하여 그날 일정이 하나도 없을 경우 이후 단계(Gmail 발송)가

	실행되지 않도록 조건 분기를 구현함
--	---------------------

3.2 프로젝트 2 요구사항 대비 구현 현황

요구 항목	구현 내용
반복 업무 1 개 정의	매일 아침 스마트스토어 운영 업무 일정을 정리해 이메일로 받아보기
도구 선정 및 이유	Make.com 선정. Google Calendar·Gmail 연동 모듈을 기본 제공하고, 무료 플랜(월 1,000 회 실행) 만으로 1 일 1 회 실행이 충분히 가능하며, 클릭 기반 UI 로 초보자도 데이터 흐름을 시각적으로 구성할 수 있기 때문
자동 실행 구조 구현	Google Calendar(Search Events) → Set variable → Text Aggregator → Gmail 순서로 연결된 시나리오를 Make.com 에 실제 구현하고 'Run once'로 다회 검증. 일정이 없는 날에는 발송이 자동 차단되도록 조건 분기도 함께 구현
워크플로우 흐름 설명 포함	4 장(과제 수행 과정)에 트리거→데이터 가공→조건 분기→발송까지의 흐름과 근거를 단계별로 서술

4. 과제 수행 과정

4.1 도구 및 트리거 선정

Make.com 에 처음 접속하여 시나리오를 생성한 뒤, 어떤 앱으로 자동화를 시작할지 검토하였다. Google Sheets·Google Docs 등 즐겨찾기 앱을 먼저 살펴보고, '스케줄러(Scheduler)' 계열 앱도 검색해보았으나 예약 서비스용 앱이라 목적에 맞지 않아 제외하였다. 최종적으로 이미 사용 중인 Google 캘린더의 일정을 그대로 활용하기로 하고 'Google Calendar - Search Events' 모듈을 트리거 겸 데이터 소스로 선정하였다.

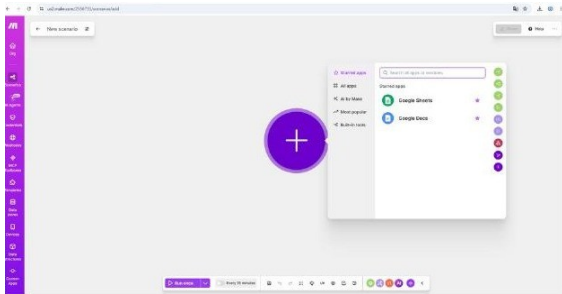


그림 1. 앱 선택 화면에서 자동화에 쓸 앱 탐색

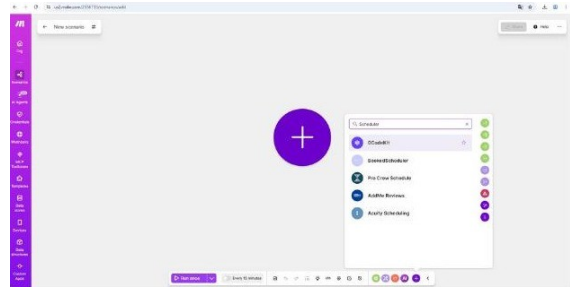


그림 2. '스케줄러' 계열 앱 검색 및 검토

4.2 Google Calendar 조회 조건 설정

Search Events 모듈에서 조회 시작일(Start Date)과 대상 캘린더를 지정하였다. 처음에는 Limit(조회 개수)을 200 으로 두고 테스트하여 캘린더 내 전체 일정 (44 건)이 그대로 반환되는 것을 확인한 뒤, 하루 단위 안내에 맞도록 Limit 을 10 으로 조정하였다. 또한 Start Date 를 고정된 날짜로 입력하면 다음 날부터는 매번 날짜를 수정해야 하므로, addDays 함수를 이용해 시작일을 자동으로 계산하도록 구성하였다.

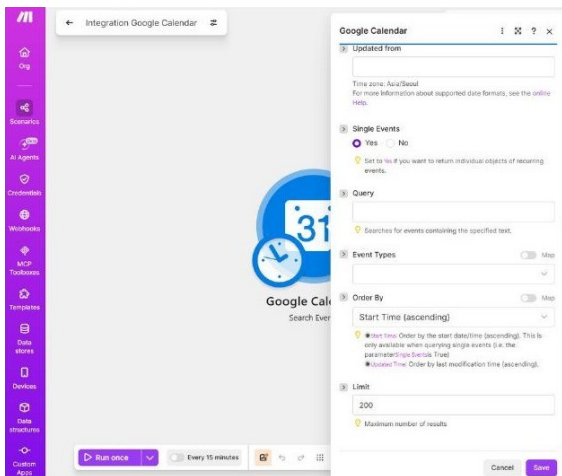


그림 3. Search Events 조회 조건(Limit 200 으로 최초 테스트)

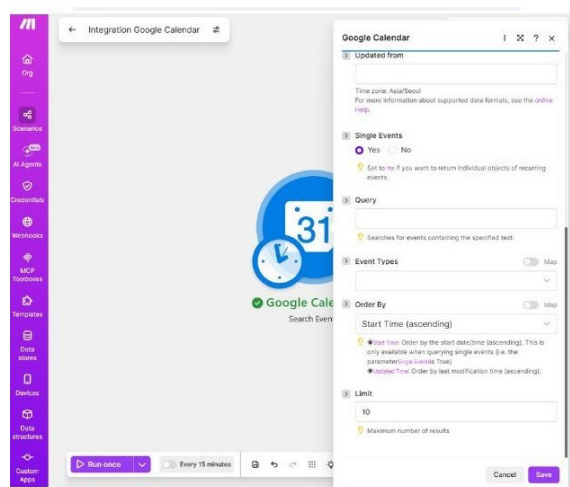


그림 4. 하루 일정 범위에 맞춰 Limit 을 10 으로 조정



그림 5. addDays 함수로 조회 시작일 자동 계산 시도

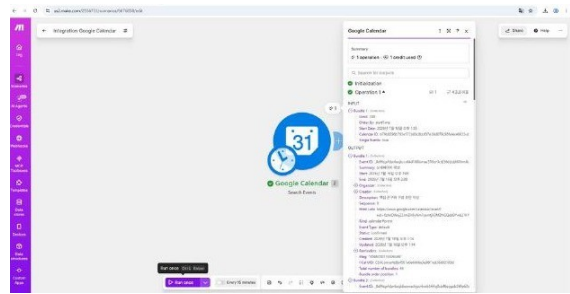


그림 6. Run once 실행 결과 - 총 44 개 일정 반환 확인

4.3 일정 병합 및 이메일 발송 모듈 연결

조회된 일정을 하나의 텍스트로 합치기 위해 Text Aggregator 를 추가하고, Summary(제목)·Start(시작 시각) 등을 병합 대상으로 지정하였다. 이어서 Gmail 'Send an email' 모듈을 연결하여 수신자, 제목('일일업무계획')을 입력하고 본문(Content)에 Text Aggregator 의 결과값을 매핑하였다. 이 과정에서 Make 는 '모듈 결과를 사용하지 않고 있다', '본문 값이 비어 있다'와 같은 경고 메시지를 통해 누락된 설정을 알려주었고, 이를 하나씩 확인하며 연결을 완성하였다.

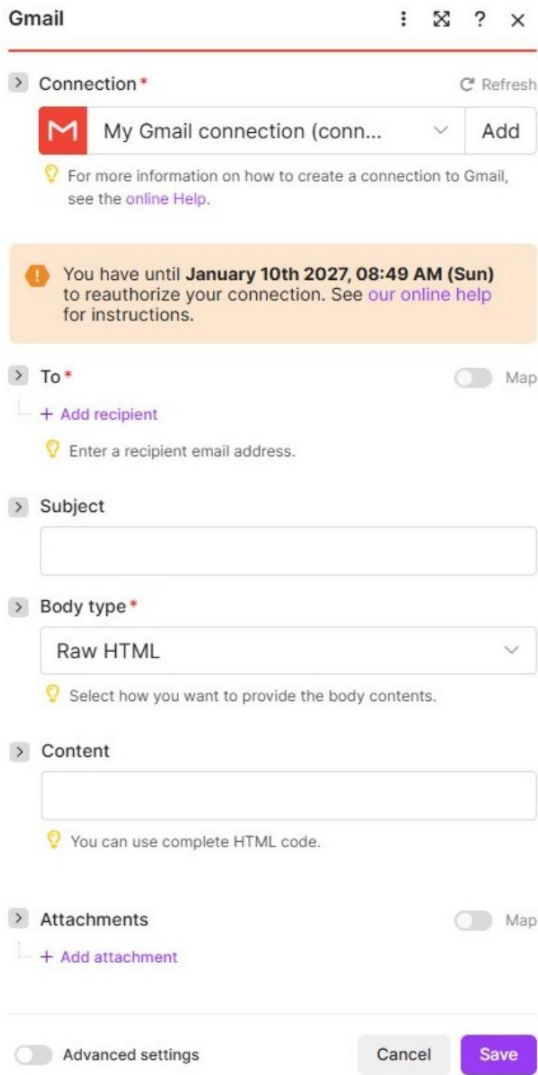


그림 7. Text Aggregator 추가 및 병합 대상 필드 지정

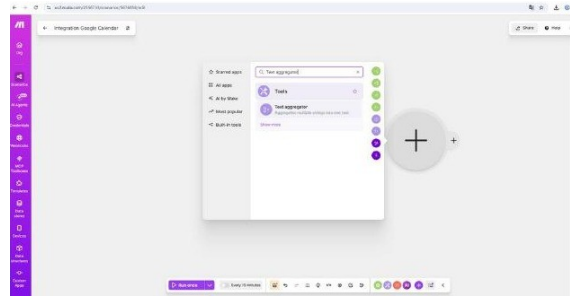


그림 8. Gmail 모듈 추가 및 수신자·제목·본문 입력

4.4 조건 분기 구현 - 일정이 없는 날 메일 발송 차단

과제 요구사항에는 조건 분기(Filter 또는 Router) 모듈을 활용해 보라는 항목이 포함되어 있었다. 처음에는 '그날 일정이 없으면 메일을 보내지 않는다'는 조건을 구현하기 위해 Gmail 모듈 앞에 별도의 Filter 모듈을 추가하려고 하였다. 그런데 이미 사용 중이던 Tools(4 번, Text Aggregator) 모듈의 고급 설정(Advanced settings)을 살펴보면, 'Stop processing after an empty aggregation'이라는 옵션이 존재하는 것을 발견하였다.

이 옵션은 병합할 데이터(그날의 일정)가 하나도 없을 때 이후 모듈의 실행 자체를 중단시키는 기능으로, 별도의 Filter 모듈을 새로 추가하지 않고도 원하는 조건 분기를 그대로 구현할 수 있었다. 이 옵션을 'Yes'로 체크한 뒤, 일정이 등록되어 있지 않은 날짜를 대상으로 Run once 를 실행해 본 결과 Gmail 발송 단계가 실행되지 않고 시나리오가 종료되는 것을 확인하였고, 실제로 해당 날짜에는 메일이 도착하지 않는 것을 확인하였다.

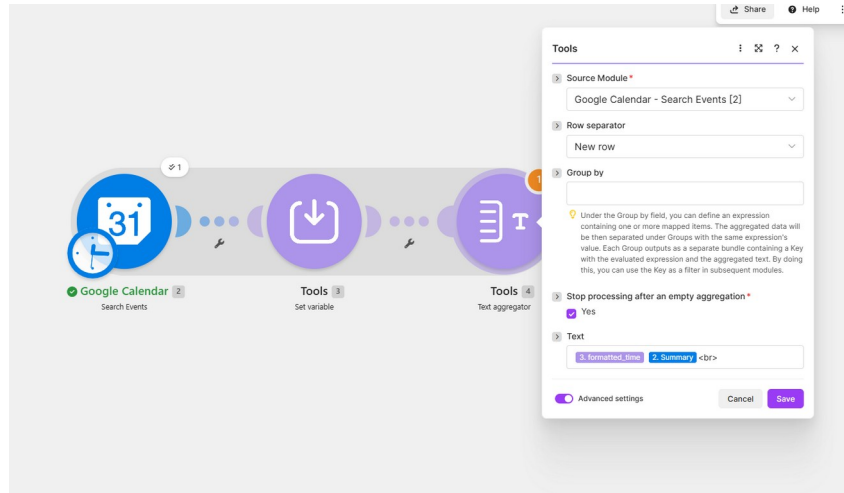


그림 9. Text Aggregator의 'Stop processing after an empty aggregation' 옵션 체크 - 일정 없는 날 발송 차단 확인

4.5 시간 표시 오류와 시행착오

전체 연결 후 실행한 결과, 수신 메일에는 시각이 '2026-07-16T07:00:00.000Z'와 같은 UTC(협정 세계시) 원본 형식 그대로 노출되었다. 한국 시간(KST)보다 9 시간 느린 값이 그대로 보여, 실제로 오후에 잡힌 일정이 오전 시각처럼 보이는 문제가 있었다.



그림 10. 1 차 시도 - UTC 원본 시각이 그대로 노출된 메일

이를 해결하기 위해 formatDate 함수로 'HH:mm' 형식과 'Asia/Seoul' 시간대를 지정하는 방법을 시도하였다. 그러나 Make 의 최신 UI 에서는 Text Aggregator 의 Text 입력창에 직접 함수식을 타이핑하는 대신, 우측 변수 패널의 항목(토큰)을 클릭해 조합하는 방식만 허용되어 수식이 그대로 적용되지 않거나, 적용되더라도 'PM0ipm/04ou7...'와 같이 낱자 형식이 뒤섞여 깨지는 결과가 반복되었다.

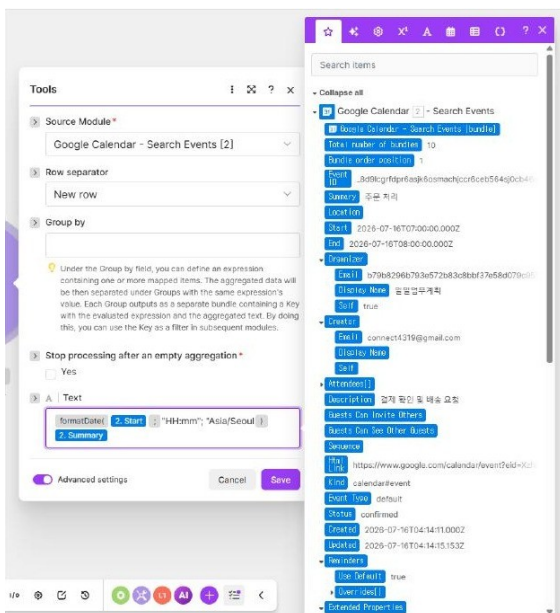


그림 11. Text 입력창에 formatDate 수식을 직접 조합해 본 시도

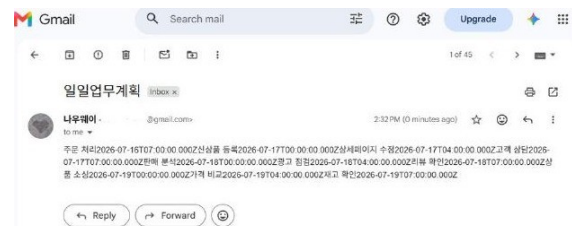


그림 12. 2 차 시도 실패 - 낱자 형식이 뒤섞여 깨진 메일

여러 차례 Row separator, Group by 옵션을 바꿔가며 원인을 좁힌 결과, Text Aggregator 자체는 formatDate 같은 가공 함수의 실행 결과가 아니라 다른 모듈이 만들어 둔 값만 그대로 이어붙일 수 있다는 한계를 확인하였다. 이에 따라 Text Aggregator 앞에 'Set variable'(Tools) 모듈을 추가하여, 이 단계에서 먼저 formatDate(Start; "HH:mm")로 시각을 'formatted_time'이라는 변수로 미리 계산해두고, Text Aggregator 는 이 변수와 Summary 만 이어붙이도록 구조를 변경하였다. 이 우회 방법을 적용한 뒤에야 원하는 'HH:mm 제목' 형식이 안정적으로 만들어졌다.

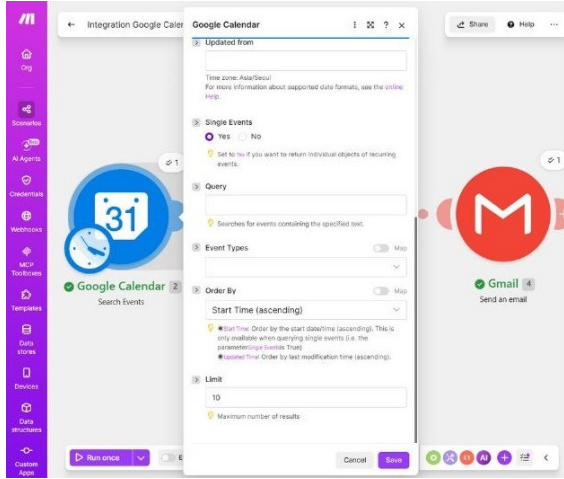


그림 13. Set variable 도입 전 - 형식이 무너진 또 다른 실패 사례

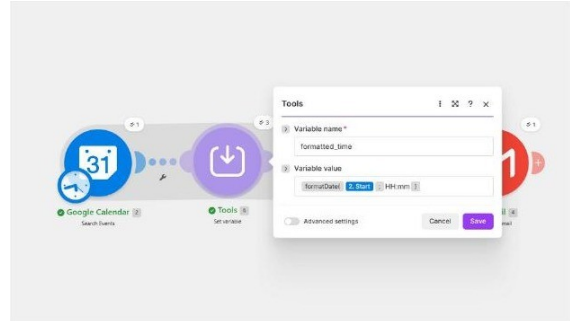


그림 14. 해결책 - Set variable 로 시간을 먼저 'HH:mm'으로 변환

4.6 오류 해결 과정 요약

증상	원인	해결 방법
메일에 UTC 원본 시각 (...T07:00:00.000Z) 노출	Google Calendar 가 반환하는 Start 값이 UTC 기준이며 별도 변환 없이 그대로 병합됨	formatDate 함수로 'Asia/Seoul' 시간대 및 'HH:mm' 형식 지정 필요성 확인
Text Aggregator 에 수식 입력 시 형식이 깨짐	최신 UI 의 Text Aggregator 는 변수(토큰) 삽입만 지원, 함수식 직접 실행 결과를 담지 못함	Text Aggregator 앞단에 Set variable(Tools) 모듈을 추가해 시간 변환을 선(先)처리
Gmail 본문이 비어 있다는 경고	Text Aggregator 결과값을 Gmail Content 항목에 매핑하지 않음	Content 필드에 '3. Text Aggregator - text' 토큰을 직접 선택하여 연결

5. 최종 결과물

최종적으로 Google Calendar(Search Events) → Tools(Set variable) → Tools(Text Aggregator) → Gmail(Send an email)의 4 단계로 구성된 워크플로우를 완성하였다. 매일 실행 시 당일 일정을 조회하여 'HH:mm 업무명' 형식으로 정리한 뒤 지정된 Gmail 주소로 자동 발송한다. 다만 그날 일정이 하나도 없을 때는 Text Aggregator 의 'Stop processing after an empty aggregation' 옵션에 의해 Gmail 단계가 실행되지 않아 메일이 발송되지 않는다.

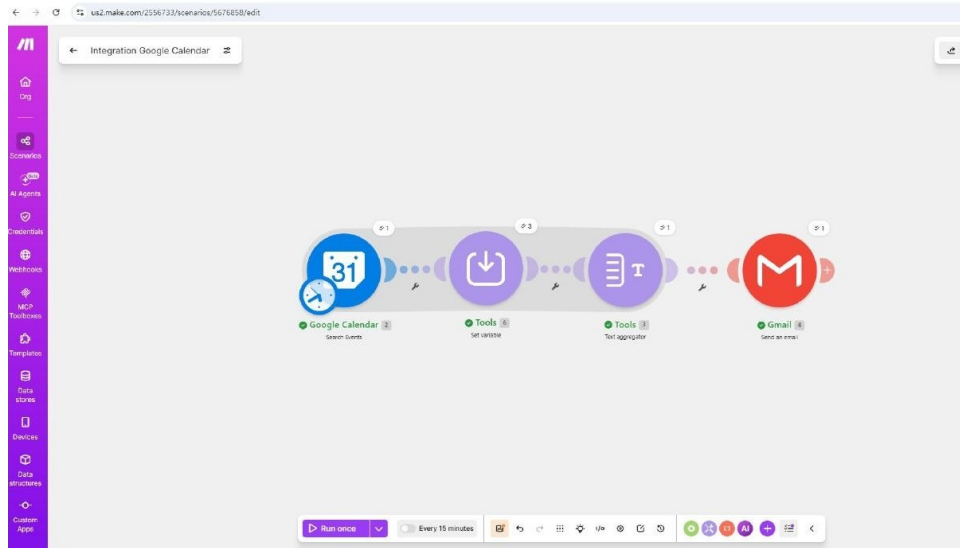


그림 15. 최종 워크플로우 구조 (4 단계 연결)

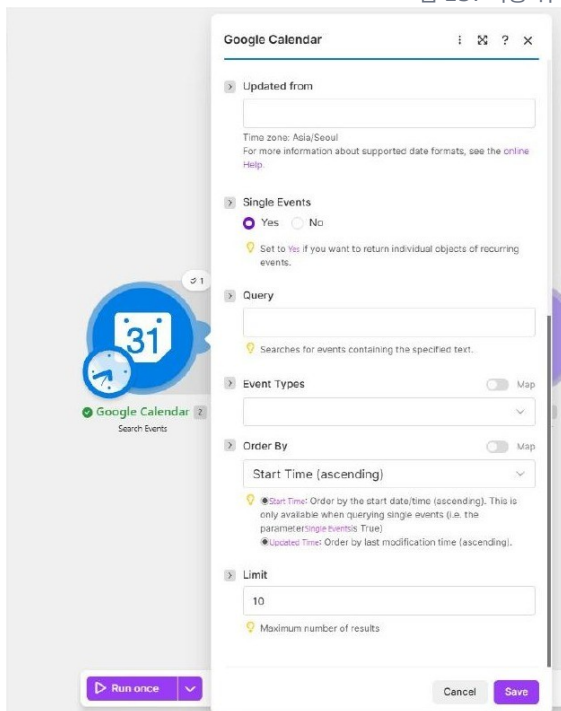


그림 16. 최종 Google Calendar 조회 조건

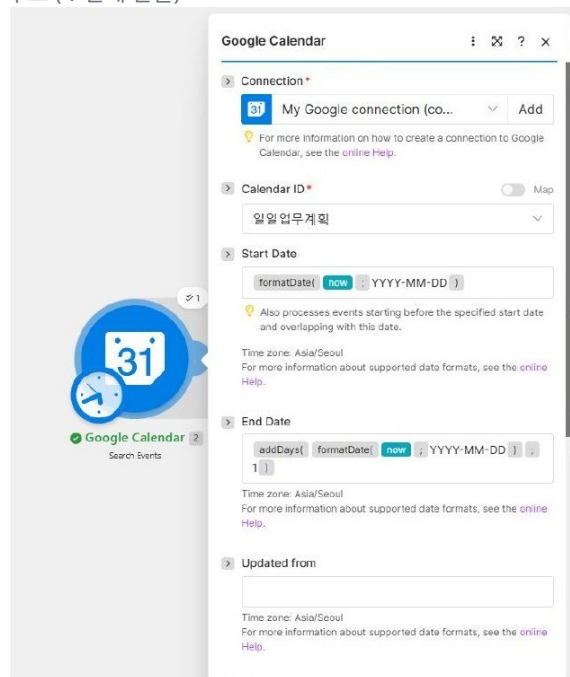


그림 17. 최종 Search Events 세부 옵션

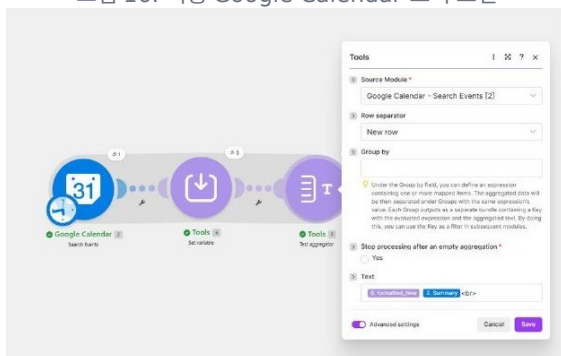


그림 18. 최종 Text Aggregator 설정 (formatted_time + Summary, 빈 일정 시 처리 중단 옵션 포함)

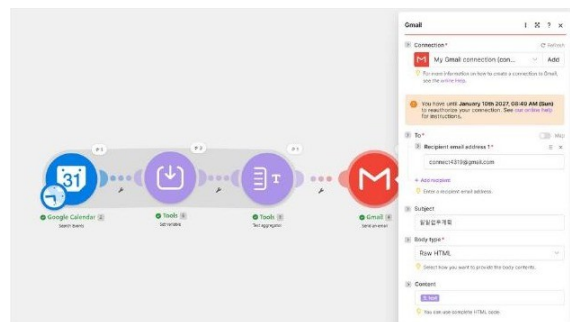


그림 19. 최종 Gmail 발송 모듈 설정

아래는 최종 워크플로우를 실행하여 실제로 수신한 이메일이다. UTC 원본 시각이나 형식 깨짐 없이, 처음 목표로 했던 'HH:mm 업무명' 형태로 하루 일정이 정리되어 도착한 것을 확인하였다.

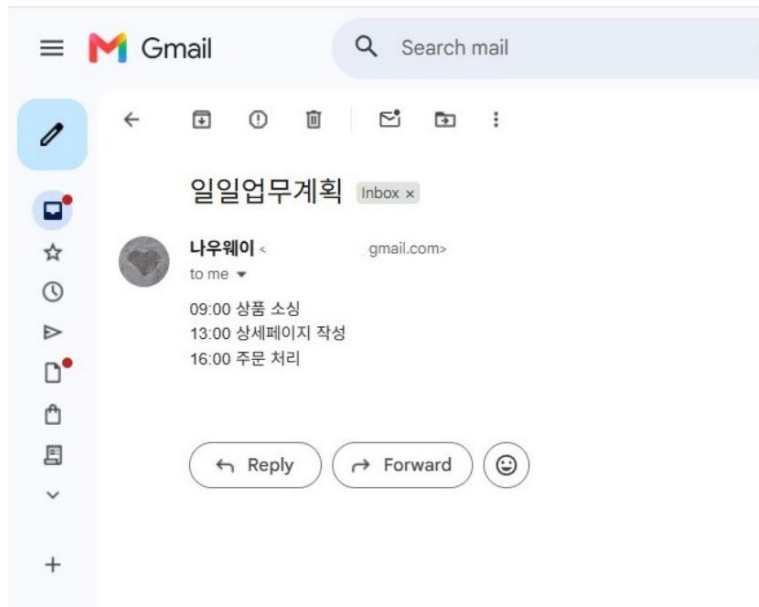


그림 20. 최종 수신 이메일 - '09:00 상품 소싱 / 13:00 상세페이지 작성 / 16:00 주문 처리' 정상 수신

6. 결론

이번 과제를 통해 Google 캘린더의 일정을 자동으로 조회하여 이메일로 전달하는 자동화를 실제로 구현하였다. 단순히 모듈을 연결하는 것을 넘어, UTC 와 KST 의 시간대 차이, Text Aggregator 가 함수 실행 결과를 직접 담지 못하는 UI 제약 등 실제로 도구를 사용하지 않으면 알기 어려운 문제들을 여러 차례의 실행과 오류 메시지 확인을 통해 스스로 해결하였다. 특히 '병합 도구가 안 되면, 그 앞에 값을 미리 계산해두는 도구를 하나 더 둔다'는 우회 방식은 이후 다른 자동화를 설계할 때도 활용할 수 있는 경험이었다.

또한 조건 분기처럼 별도 모듈이 필요할 것 같은 기능도, 새 모듈을 추가하기 전에 이미 쓰고 있는 도구의 옵션을 먼저 꼼꼼히 살펴보는 것이 더 효율적일 수 있다는 점을 확인하였다. Text Aggregator 의 'Stop processing after an empty aggregation' 옵션 하나로 '일정이 없는 날은 메일을 보내지 않는다'는 조건 분기를 별도 Filter 모듈 없이 구현할 수 있었던 것이 대표적인 사례이다.